A Code-First Approach to Teaching Software Design Patterns

Jeffrey Hemmes
Department of Computer and Cyber Sciences
United States Air Force Academy
jeffrey.hemmes@afacademy.af.edu

***Abstract*—Software design patterns are an important element of professional software engineering practice, and therefore a topic typically covered in undergraduate software engineering courses as part of a computer science curriculum. Despite the importance of design patterns in both professional software development and computer science education, best pedagogical practices are very frequently not applied when teaching them, with a traditional lecture-based approach still commonly used. Student-centered learning involving hands-on activities and self-discovery of lesson topics is a proven approach towards engaging and effective teaching and learning. In this paper, a pedagogical methodology for minimizing the extent to which instructor-led lecturing is used in teaching software design patterns and replaced with student-focused learning activities that increases engagement and student interest is presented. The methodology involves taking a comprehensive code-first or bottom-up approach to teaching what is all to frequently a tedious and repetitious subject to cover in class lessons.**

I. Introduction

Undergraduate survey courses in software engineering typically include blocks of instruction covering design patterns. The topic is important because it represents a significant next step beyond simply learning programming language syntax, algorithms, and data structures, and towards development of scalable software-intensive systems that are understandable, maintainable, and reusable. Software design patterns are an important tool in professional software development to help organize code, manage complexity, and provide flexibility and maintainability, particularly in larger or more critical systems. Design patterns also provide a common vocabulary with which software engineers communicate complex design concepts in an efficient and shorthand manner. They represent solutions to commonly recurring design problems, and recognizing those problems as they commonly manifest allows students to work more efficiently by avoiding having to solve problems that have already been solved by others.

Teaching design patterns as part of an undergraduate course poses two challenges to course designers and instructors. First, although they represent an important concept to apply in professional practice, the majority of students taking a software engineering course as part of an undergraduate program typically lack programming maturity and experience in software development, which has been a longstanding observation in this context [2]. Consequently, it is uncommon for novice students to relate to the types of design problems patterns solve. Most typically, undergraduate software engineering courses are taken during a student's junior year, or perhaps fall semester of their senior year. By that time, students often have completed only a handful of computer science courses, and most likely have not worked on a larger-scale software development effort. This is almost paradoxical: students need to quickly understand and apply concepts related to professional software development practice, while simultaneously lacking the experience and conceptual knowledge to do so. The all-too-frequent result is that design pattern concepts remain abstract and unrelatable to students, and they do not internalize the material as much as instructors may hope.

The second challenge in teaching design patterns to undergraduate students lies in the nature of the material. Going back to the Gang of Four catalog, design patterns have always been organized in a structured taxonomy with three primary categories: creational patterns; structural patterns; and behavioral patterns. Each category then contains a collection of patterns. The most common, traditional approach to teaching design patterns involves presenting them in a dry, methodical manner heavy on lecture and mirroring the structure of the Gang of Four taxonomy. For each pattern the general design problem is presented, followed by the abstract solution model, and then perhaps lastly some example code in an object-oriented programming language. Beyond the original Gang of Four, catalogs of additional patterns are introduced with some regularity as new problems and associated designs arise, e.g., [1], resulting in a significant volume of catalogued design patterns.

The latter challenge represents a more significant barrier to the effective teaching of design patterns: doing so is boring, repetitious, and not engaging to students. Despite the importance of the topic, students often have difficulty learning and retaining many of the details and nuances of various patterns and might retain at best only basic information, such as the general nature of a Singleton pattern, for instance. It has long been acknowledged that design patterns are "both difficult to learn and teach." [15].

Part of the reason for this difficulty is a tendency to focus on the abstract before the implementation (a significant tenet

of object-oriented design and programming, which itself is a challenging topic for many students [10]). However, without the requisite background and maturity in programming, the abstract concepts tend to be difficult to grasp.

With that in mind, what if we flip the script and start with the familiar, then work up incrementally to new concepts? In other words, how can we take a bottom-up approach to teaching design patterns and making it more engaging than a standard, traditional lecture?

II. Related Work

Despite decades of community experience in teaching software design patterns to undergraduate computer science students and a significant number of publications regarding the use of patterns in professional practice [13], there is a dearth of literature describing current pedagogical best practices in this topic area. This is despite the substantive differences in covering design patterns compared to other software engineering topics. Beacuse of this, teaching design patterns is an area of software engineering education that can be improved and the application of student-centered pedagogy specifically to design pattern lessons is worthy of examination. Indeed, experience has shown that traditional lecture-based approaches to teaching patterns based on longstanding practices such as those described in [5]–[7], [18], [22] are still commonplace. This section provides a brief survey of the literature reflecting current pedagogical practice.

Since the introduction of the Gang of Four's Design Patterns catalog [8], patterns have been covered in undergraduate programming courses, even as early as CS1 [20]. Explaining the benefits of using patterns to create elegant, maintainable software designs similarly has a lengthy history [16] and is commonly emphasized in lesson materials as motivation.

One common practice is to provide students with an existing program of some size that either contains patterns fully implemented [4], or does not, but provides opportunities to apply them, e.g., [3]. Examining pre-written code to determine appropriate applications of design patterns is certainly nothing new. Commonly we see this approach with examples taken from computer gaming [9], [11], [19]. These types of programs tend to be rather large with some inherent complexity. Games also tend to be designed and implemented using multiple logical layers representing separation of concerns, e.g., graphics, game play, data, etc., and therefore there is an abundance of opportunity for students to discover areas to improve and modify the design and implementation through application of patterns. However, to do so effectively, students must already be familiar with some subset of the catalog of design patterns. But how do they learn that catalog in the first place? Ultimately, even current practice for presenting patterns often involves an instructor lecturing over the catalog of patterns, stepping through the the design problem for each pattern, and presenting the abstract model, typically using the Unified Modeling Language (UML).

III. Methodology

The instructional model we use to teach design patterns involves four sequential steps, each building on the previous, serving a distinct purpose, and being focused on student engagement through hands-on learning. These steps are intended to provide student-centered instruction that minimizes the time instructors spend lecturing over the abstract pattern designs.

A. Step 1: Examine a Working Code Example

To begin, students are provided a working example in code that implements a common design pattern. They are not told yet what pattern it is or what design problem it solves, as this step is intended to serve as a discovery process. Students are initially asked to examine the code and explain what they think it does. They may run the program as well if they so choose and study output or program behavior. They are also asked to identify various elements within the code, e.g., concrete and abstract classes, interfaces, inheritance, polymorphism, abstractions, and anything else that stands out. This can include unfamiliar concepts or anything that appears unusual, which may be any modularity or extensibility features present in the design. They are also asked to comment on how easily understandable the design and implementation are. When covering design patterns, students already have the prerequisite familiarity with object-oriented concepts and object-oriented programming, although they may not have fully achieved mastery at this point.

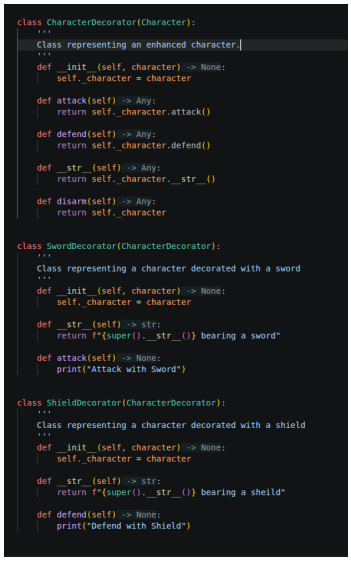
The examples provided for study are drawn from problem domains with which students have familiarity, e.g., video game characters, smartphone capabilities, food delivery apps, etc. This better allows them to focus on how the various code elements work together to provide that functionality by minimizing the number of unknown or unfamiliar aspects to which students are exposed. An example of an object-oriented Python program snippet that implements a Decorator pattern using a video game character is shown in Figures 1 and 2.

```python
class GameElement(ABC):
    '''
    Abstract class representing a generic game element
    '''
    @abstractmethod
    def attack(self) -> None:
        pass

    @abstractmethod
    def defend(self) -> None:
        pass


class Character(GameElement):
    '''
    Concrete class representing a character
    '''
    def attack(self) -> None:
        print("Attack with bare hands")

    def defend(self) -> None:
        print("Defend with bare hands")

    def __str__(self) -> Literal['Base character']:
        return "Base character"
```

Fig. 1. Decorator Pattern Abstract Base Classes. Class hierarchy implementing elements of a Decorator pattern in Python using the example of game characters that can be augmented with new capabilities for attack and defense.

B. Step 2: Create Abstract Model from the Code

Once students understand what the code example does and have identified the relevant design elements, they are asked



```
class CharacterDecorator(Character):
    '''
    Class representing an enhanced character.
    '''
    def __init__(self, character) -> None:
        self._character = character

    def attack(self) -> Any:
        return self._character.attack()

    def defend(self) -> Any:
        return self._character.defend()

    def __str__(self) -> Any:
        return self._character.__str__()

    def disarm(self) -> Any:
        return self._character


class SwordDecorator(CharacterDecorator):
    '''
    Class representing a character decorated with a sword
    '''
    def __init__(self, character) -> None:
        self._character = character

    def __str__(self) -> str:
        return f"{super().__str__()} bearing a sword"

    def attack(self) -> None:
        print("Attack with Sword")


class ShieldDecorator(CharacterDecorator):
    '''
    Class representing a character decorated with a shield
    '''
    def __init__(self, character) -> None:
        self._character = character

    def __str__(self) -> str:
        return f"{super().__str__()} bearing a sheild"

    def defend(self) -> None:
        print("Defend with Shield")
```

Fig. 2. Decorator Pattern Concrete and Decorated Classes. Example concrete decorator classes implementing additional attack and defense capabilities provided to a game character.

to create an abstract model of the code using a UML class diagram. By doing so, they can better visualize the elements and how they relate to each other as an object-oriented model. Students also visualize the purpose of class instance variables and how they contribute to program functionality when implemented in code.

The model students care as part of this step is an abstract representation of a particular problem solution. When creating this model, students are asked to think about how the abstract elements in the model can be further generalized to make the solution less application specific. The instructor's role is to further reinforce understanding of the design elements and critique the models. Student peer review can also be used as part of this activity.

C. Step 3: Present the General Pattern

Once students have had hands-on experience with the code and created an abstract model of that code, the more traditional design pattern lesson begins. However, because of the prerequisite activity work, this lesson is abbreviated as no new information needs to be introduced. Taking the application-specific design models as a starting point, the general design pattern model can be easily derived. The identification and purpose of model elements, the object-oriented design concepts used, and the relationships between elements have already been established. What is traditionally the most tedious and unengaging part of teaching design patterns can be completed in a matter of a few minutes.

D. Step 4: Apply to a New Problem

In the final step, a more complex program is given to students, much like as suggested by Nevison [14], Wallingford [20], and others. This program does not contain any design patterns, but provides an opportunity for refactoring. As the last portion of the lesson, students identify how the pattern covered might fit into the existing program. They identify the application-specific classes that correspond to the abstract general model and create a new design for the program that includes the pattern. This gets to the heart of what the design pattern is and why it exists, i.e., what problem it is intended to solve. By taking this step, students gain an appreciation for the need and benefit of such patterns.

In this step, the analysis of the flawed program is part of an instructor-guided discussion. Part of the learning involves not only the identification of program segments that can be improved using design patterns, but a discussion and understanding of what the code lacks, how an appropriate design pattern can improve readability, simplicity, or extensibility, and how that pattern would fit into the existing program structure.

For the example in Figures 1 and 2, students map the elements of the Game Character class to the abstract components of the Decorator pattern. In particular, the relatable `attack` and `defend` operations are abstracted into the general `operation` method in the canonical pattern design, as shown in Figure 3.

IV. Implementation and Feedback

This approach was applied in an undergraduate software engineering course in a single lesson covering the topic of design patterns ($n = 41$). That introductory lesson was subsequently followed by a practical application lesson that allows more open-ended problem solving activities. The course enrollment included students not only in the computer science major, but also in cyber science, systems engineering, and data science programs. Student feedback on the lessons was overwhelmingly positive. This is a substantial contrast to previous years' offerings in which a more traditional approach was taken. Historically, students generally feel that a traditional approach goes too quickly, lacks ability to work with patterns in a hands-on and timely manner, and tends to be dry and unengaging. These concerns were corrected by the code-first approach, based on student feedback.

V. Discussion

This work is not intended to necessarily increase student proficiency in understanding and applying design patterns. The intent is more to apply student-centered pedagogy to a topic that frequently lacks engagement in practice. By starting from a code example of a design pattern implementation that leads to the more abstract model, the conceptual basis for the pattern, and the design problems it is intended to solve, we minimize to the extent possible classroom time spent in lecture covering unrelatable and repetitious topics.

With a more methodical approach, this methodology also prevents one of the most common problems instructors face when covering design patterns: trying to cover too many. Needless to say, one cannot, and would never wish to, teach a large number of patterns. However, one common pitfall is for instructors to attempt to power through a large number of patterns, resulting in little or no student retention. By spending a

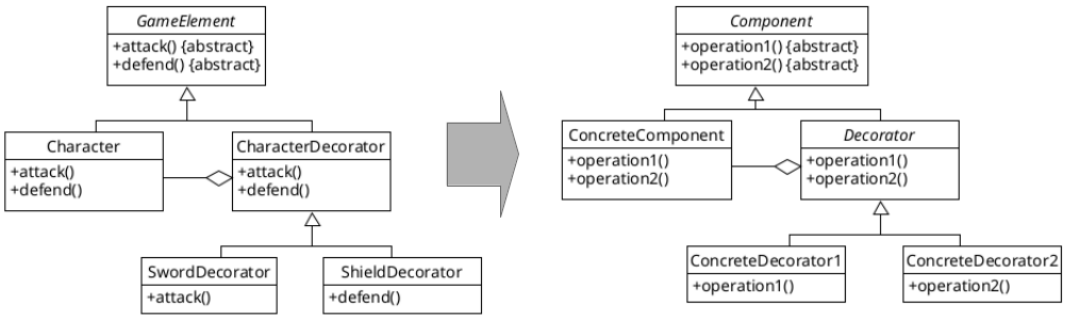


Fig. 3. Decorator Pattern Class Mapping. Students are asked to transform an application-specific model of an implemented design pattern into a similar model of the general pattern through a hands-on activity involving the identification of abstract design elements present in an existing code example.

bit more time on each one and allowing student self-discovery, the number of distinct patterns taught is kept minimal (perhaps at most two or three patterns per category), focusing primarily on those patterns most commonly appearing in practice and that solve problems students are likely to have encountered in programming exercises during earlier course work.

The discovery portion allows students to better see firsthand the benefit of applying patterns and the specific design patterns they are intended to solve. If students first encounter a code example that solves a real and relatable problem, they can appreciate the motivation behind and the purpose for the pattern. When the abstract pattern is introduced afterward, it appears as an incremental generalization of something they have already experienced rather than as an arbitrary diagram or UML structure that must be parsed and digested as an unfamiliar whole. For patterns not covered during a lesson, one takeaway for students is an understanding that patterns exist and to reference a catalog of patterns if they are stuck on a particular design problem.

The argument in favor of a code-first approach is that students start with the small and familiar, then incrementally work upwards to the larger and more abstract.

Teaching design patterns in such a manner requires some prerequisite knowledge. First, students must have some background and comfort level with object-oriented design concepts and object-oriented programming. Students must also have familiarity with UML class diagrams for modeling and discussion of patterns. Using Python as an object-oriented programming language works well as it produces concise and readable code that matches well with their corresponding abstract UML design elements. In our experience, placing the topic approximately one-third of the way into a semester schedule is a suitable placement, and taking two or three lessons to cover a small number of significant patterns generally works well.

Using a bottom-up approach for design patterns is not incompatible with other topics covered in a software engineering course. It is common to cover software testing, test-driven development, agile software development processes, and other topics using a similar approach. Because of this consistency, students have not been confused by the methodology or thought the pedagogy was something of an anomaly within the context of the overall lesson flow.

VI. Conclusions and Future Work

This paper presented an approach intended, not necessarily to measurably increase student proficiency on assessments related to the topic of software design patterns and their application, but instead to make the lesson coverage of the topic more student-centered, hands-on, and engaging than it current practice suggests. To do this, we propose taking a bottom-up approach with a code example, then working up to the more abstract generalizations of each pattern covered. This allows students to start with small-scale, familiar, and easy-to-understand examples and incrementally work up to the abstract models and the associated problems. This minimizes the amount of instructor-centered lecture coverage needed to present new material and leaves students with far more activity-focused lessons.

Despite the increase in student-centered activity, several issues related to the teaching of design patterns remain unresolved. These issues are what Köppe described as pitfalls related to the teaching of patterns [12]:

- Students often attempt to apply patterns too hastily. It is natural to assume that if one identifies a place where a pattern might fits, refactoring code a pattern there doesn't necessarily improve the code. Knowing when not to apply a pattern is an equally important skill for students to develop.
- Insufficient experience to see design patterns' advantages. Although the pedagogy described in this paper provides initial experience working with patterns, fully understanding the benefits of design patterns requires more experience than can be provided in an undergraduate academic setting with a limited amount of class time available while keeping student workload reasonable.
- A tendency to overdo the application of patterns. A corollary to applying patterns too hastily is applying patterns where they are not needed at all. In our experi-

ence, students have a strong tendency to overuse certain patterns such as Singleton.

- Students do not necessarily see patterns as a best practice. This is again a consequence of limited experience that can be addressed somewhat, but not fully, in an undergraduate course.

All of these pitfalls are topics to be addressed in future iterations of our software engineering and related software development and computer science courses.

REFERENCES

[1] Al-Ahmad, W., *Object-Oriented Design Patterns for Detailed Design*. Journal of Object Technology 5:2, 2006.

[2] Carrington, D., *Teaching Software Design and Testing*, Proceedings of the 28th Annual Frontiers in Education Conference (FIE '98), vol. 2, 1998.

[3] Carrington, D., *What Can Software Engineering Students Learn from Studying Open Source Software?*. International Journal of Engineering Education (IJEE) 24:4, 2018.

[4] Chimalakonda, S., and Nori, K., *A Patterns Based Approach for Design of Educational Technologies*. Interactive Learning Environments 31:4, 2021.

[5] Christensen, H., *Frameworks: Putting Design Patterns into Perspective*. SIGCSE Bulletin inroads, 36, 3 (September 2004), pp 142-145.

[6] Clancy, M., and Linn, M., *Patterns and Pedagogy*. SIGCSE Bulletin Inroads, 31, 1(March 1999), pp 37-42.

[7] Dewan, P., *Teaching Inter-Object Design Patterns to Freshmen*. SIGCSE Bulletin Inroads, 37, 1 (March 2005), pp 482-486.

[8] Gamma, E., Helm, R., Johnson, R., and Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[9] Gestwicki, P. and Sun, F., *Teaching Design Patterns Through Computer Game Development*. ACM Journal on Educational Resources in Computing (JERIC) 8:1, 2008.

[10] Hadar, I. and Leron, U., *How Intuitive is Object-Oriented Design?*. Communications of the ACM, 51:5 2008.

[11] Healy, J., *Game Design Learning: Understanding and Supporting the Game Design Process in Higher Education*. Ph.D. dissertation, School of Media, Technological University Dublin, Dublin, Ireland, 2023.

[12] Köppe, C. *A Pattern Language for Teaching Design Patterns (Part 2)*. Proceedings of the 18th Conference on Pattern Languages of Programs (PLoP '11), October, 2011.

[13] Lartigue, J. and Chapman, R. *Comprehension and Application of Design Patterns by Novice Software Engineers*, Proceedings of the Annual ACM Southeast Conference, March, 2018.

[14] Nevison, C., and Wells, B., *Teaching Objects Early and Design Patterns in Java Using Case Studies*, Proceedings of the 8th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE03), July, 2003.

[15] Pillay, N., *Teaching Design Patterns*, Proceedings of the Southern African Computer Lecturers' Association Conference (SACLA), 2010.

[16] Schmidt, D., *Using Design Patterns to Develop Reusable Object-Oriented Communication Software*. Communications of the ACM, 38:10, 1995.

[17] Sterkin, A., *Teaching Design Patterns*. Proceedings of the Frontiers in Education: Computer Science and Computer Engineering Conference (FECS), 2006.

[18] Stuurman, S., and Florijn, G., *Experiences with Teaching Design Patterns*. Proceedings of the 35th SIGCSE Technical Symposium on Computer Science Education (SIGCSE '04), March, 2004.

[19] Vahldick, A., and Vargas Nolli, J., *Introducing the Visitor Design Pattern through a Digital Serious RPG Game*

[20] Wallingford, E. *Toward a First Course Based on Object-Oriented Patterns*. Proceedings of the 27th SIGCSE Technical Symposium on Computer Science Education (SIGCSE '96), March, 1996.

[21] Weiss, S., *Teaching Design Patterns by Stealth*. Proceedings of the 36th SIGCSE Technical Symposium on Computer Science Education (SIGCSE '05), March, 2005.

[22] Proulx, V., *Programming Patterns and Design Patterns in the Introductory Computer Science Course*. SIGCSE Bulletin Inroads, 32, 1(March 2000), pp 80-84.